

基于 Conformer 的语音识别复现 与 Squeezeformer 架构优化研究

组名: 404NotFound

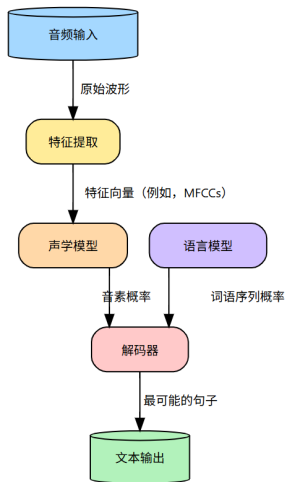
成员: 韩羽宸、李江楠、黄婉文

目录

- ① 项目背景与动机
- ② 基准模型：Conformer 架构
- ③ 深度优化：Squeezeformer
- ④ 跨平台：华为 MindSpore 迁移
- ⑤ 实验展示与数据分析
- ⑥ 总结

何为自动语音识别 (ASR)?

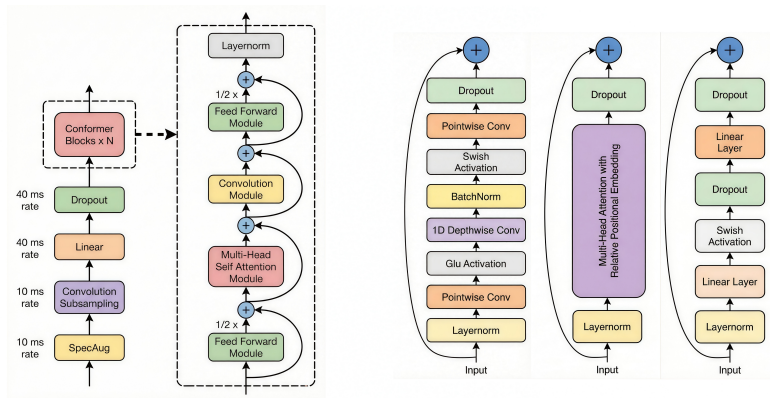
- **核心目标**：将连续的语音信号转换为离散的文字序列
- **端到端 (End-to-End)**：
 - 过去：声学模型 + 字典 + 语言模型（极其复杂）
 - 现在：一个神经网络搞定（简洁高效）
- **挑战**：
 - 局部性：音素 (Phoneme) 的变化是局部的
 - 全局性：一整句话的语义关系是长程的



标准ASR流程图。音频数据从左到右流动，在每个阶段进行转换。

Conformer: 卷积增强型 Transformer

如何设计?: Transformer 擅长抓全局, CNN 擅长抓细节, 那就将两者综合!



- 三明治结构: 两个 Feed-Forward 模块夹着 Attention 和 Convolution
- Macaron 风格: 借鉴了马卡龙的结构, 各层之间通过半步残差连接

- **多头自注意力 (MHSA):** 让模型学会 attention。引入了**相对位置编码**。
- **卷积模块 (Conv Module):**
 - 采用深度可分离卷积 (Depthwise Conv)
 - 捕捉声学信号中像“辅音/元音”这样的局部突变
- **CTC 损失函数:** 解决“对齐”问题。不需要标注每一毫秒对应哪个字，只需要知道整句话的内容

为什么要优化? (Squeezeformer 动机)

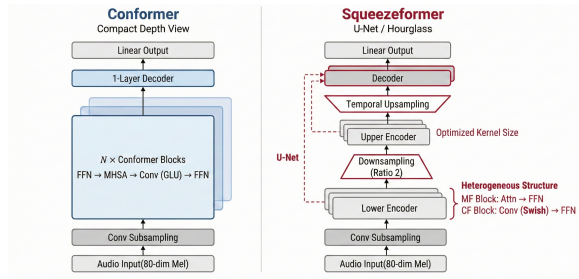
原模型缺点分析

- **计算冗余**: 语音信号在深层网络中, 相邻帧之间高度相似
- **计算开销**: 标准 Conformer 在长句子上速度较慢

我们的策略: 借鉴计算机视觉中的 U-Net 思想, 进一步精简模型

改进 1: 时序 U-Net 结构

- **下采样 (Downsampling):** 在模型中间把序列长度减半
- **计算节约:** Attention 的计算复杂度是序列长度的平方, $N \rightarrow N/2$ 意味着计算量减小为 1/4
- **上采样 (Upsampling):** 在输出前还原长度, 保持识别精度

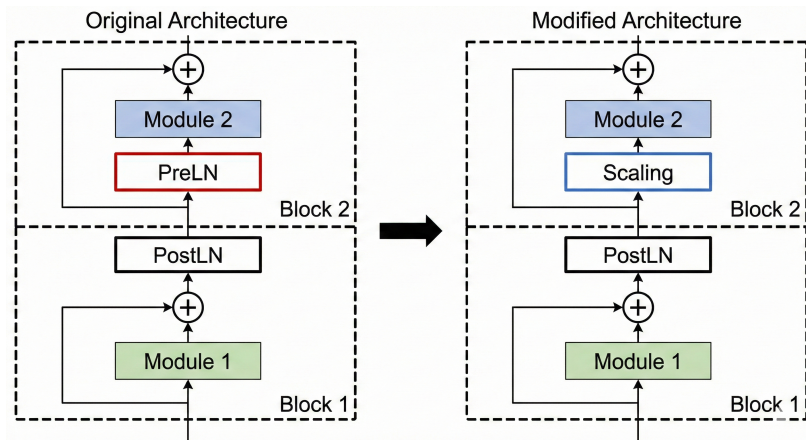


改进 2: 简化归一化 (Scaled Post-LN)

- **问题:** Conformer 混用了 Pre-LN 和 Post-LN, 导致训练时有些冗余
- **方案:** 引入一个可学习的缩放层 (Scaling Layer)

$$\text{Scaling}(x) = \gamma x + \beta$$

(1)



- 目标：将 PyTorch 代码完整迁移至 MindSpore 2.7.0
- 硬件：华为云 ModelArts + 昇腾 (Ascend) NPU

映射类别	PyTorch 实现	MindSpore 对应
基础单元	<code>nn.Module</code>	<code>nn.Cell</code>
前向传播	<code>forward()</code>	<code>construct()</code>
算子对应	<code>torch.matmul</code>	<code>ops.BatchMatMul</code>
设备设置	<code>tensor.cuda()</code>	<code>context.set_context(device_target='Ascend')</code>

迁移中的“坑”与填补方案

1. 梯度裁剪 (Gradient Clipping)

PyTorch 直接调用工具函数, MindSpore 需要手动定义 `ops.GradOperation` 并封装 `TrainOneStepCell`

2. 数据处理

`torchaudio` 与 NPU 并不兼容。我们改用更通用的 `librosa` 进行特征提取, 确保了输入数据的一致性

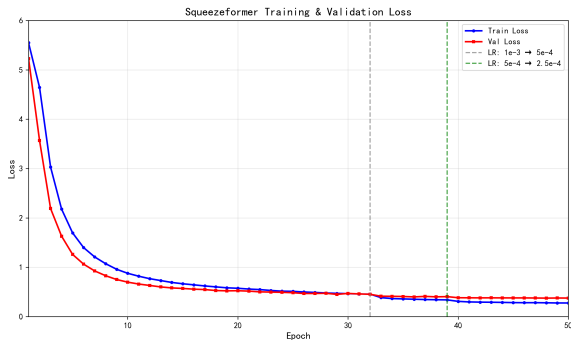
3. 权重转换

将 `.pth` 格式转换为 `.ckpt` 格式, 并确保参数名逐一对应

- 数据集: AISHELL-1 (中文开源)
- 数据量: 约 178 小时语音
- 特征: 80 维梅尔频谱
- 硬件: 昇腾 NPU 处理核心

超参数	数值
Batch Size	64
Optimizer	AdamW
Learning Rate	0.001
Epochs	30-50

训练过程监控 (Loss)



- 观察点: Squeezeformer 收敛速度较快, 模型收敛良好: Loss 从 5.5 降到 0.28
- 稳定性: 引入 Scaled Post-LN 后, 模型没有出现梯度消失或爆炸, 学习率调度有效: 在第 32 轮和第 39 轮自动衰减
- 可靠性: Train Loss 下降 95.01%, Val Loss 下降 92.80%

核心性能对比 (CER ↓)

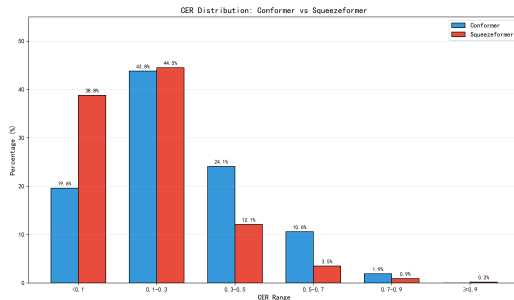
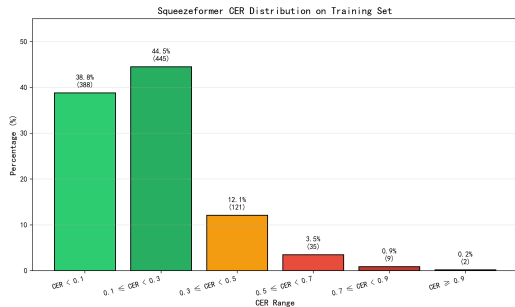
字错误率 (Character Error Rate) 是衡量语音识别的核心指标:

评估维度	Conformer (基准)	Squeezeformer (优化后)
测试集整体 CER	31.85%	24.50%
训练集整体 CER	25.56%	16.54%
识别质量提升	-	+23.1% (相对提高)

结论: 通过优化下采样和激活函数, 模型在更轻量的前提下, 识别精度大幅提升

数据深度分析：错误率分布

我们统计了所有测试样本的 CER 分布情况：



- 完美识别：CER < 10% 的样本量从 19.6% 暴增到 **38.8%**。
- 鲁棒性：极端错误（CER > 70%）的情况显著减少

案例展示：识别结果对比

Case 1: 简单句

真值：海峡都市报大白天

识别：海峡都市报大白天 (完美对齐)

Case 2: 同音字挑战

真值：他似乎在搜狐工作

Conformer：他**搜狐**在**似乎**工作 (语义错乱)

Squeezeformer：他似乎在**搜狐**工作 (上下文建模更强)

- ① **全栈复现**: 完成了从音频处理、模型搭建到训练评估的闭环
- ② **架构创新**: 成功引入 Squeezeformer 改进, 验证了 U-Net 结构在语音任务中的潜力
- ③ **国产化适配**: 突破了 MindSpore 在底层算子和训练链条上的迁移难题, 成功运行在昇腾算力平台

下一步计划

- **模型压缩**: 尝试知识蒸馏 (Knowledge Distillation), 进一步减小参数量
- **流式识别**: 优化模型使其能够边听边说, 降低实时延迟
- **引入大模型**: 结合 LLM 进行二次纠错 (Rescoring)

谢谢大家! 欢迎大家提问